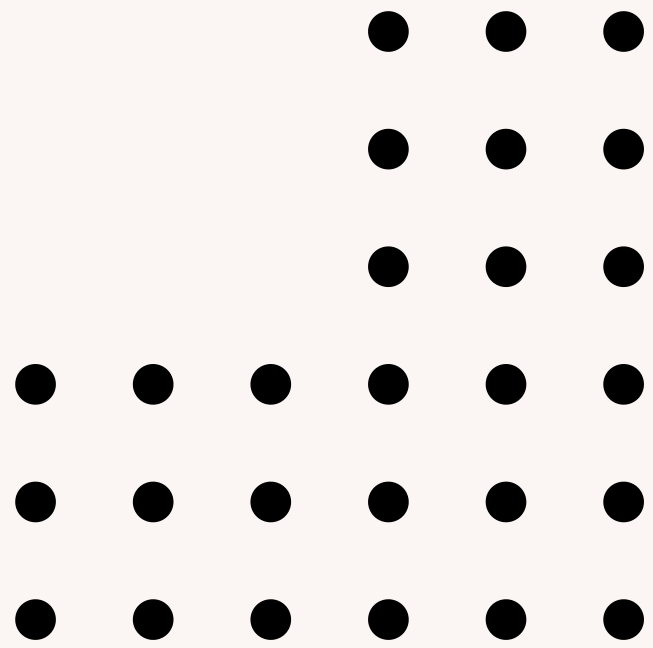
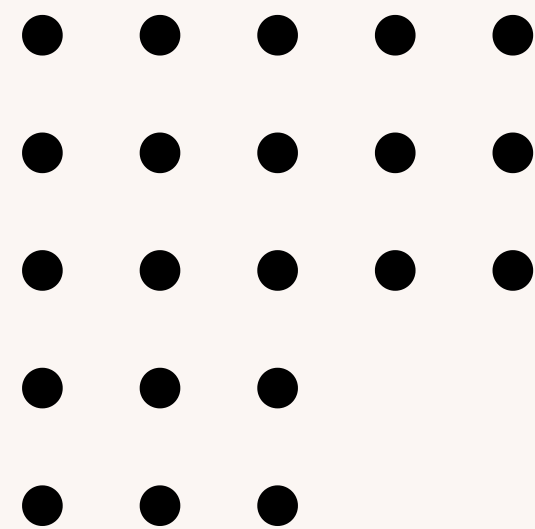




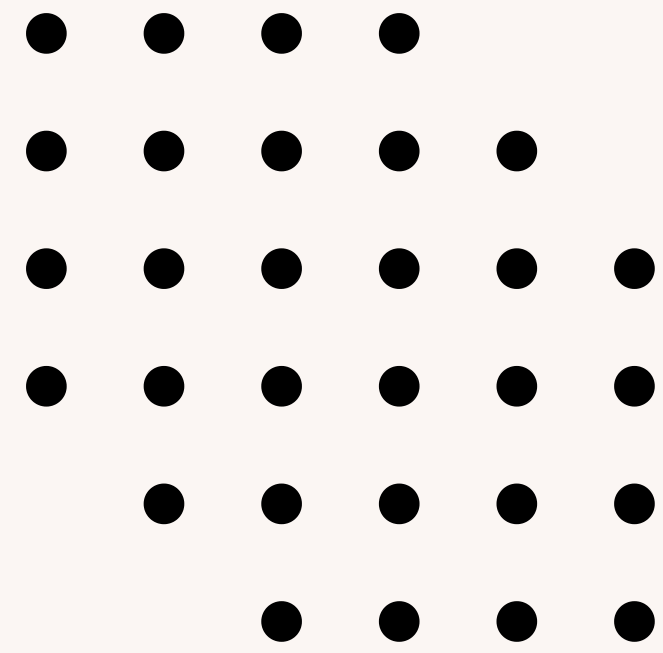
PARADIGMAS DE LINGUAGEM DE PROGRAMAÇÃO

- Grupo: Carlos Henrique Brasil, Márcio Alves,
Miguel Arcanjo, Paola Gualande, Pedro Henrique Rocha



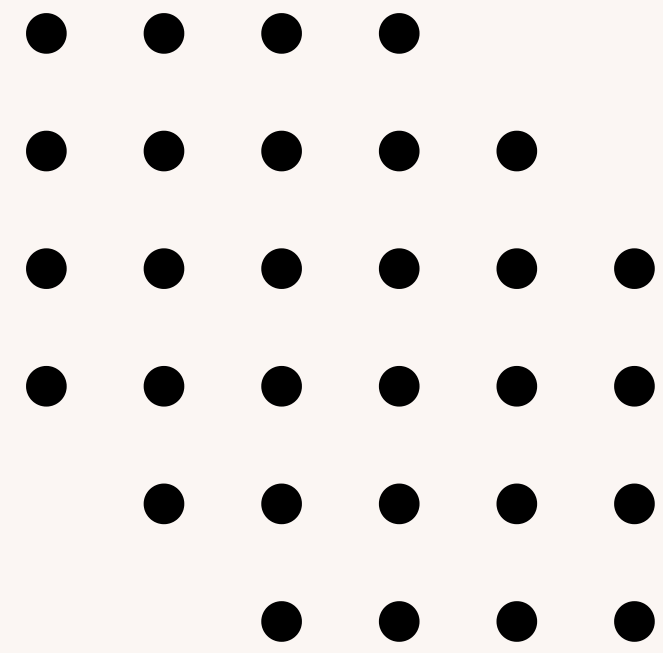
TÓPICOS

- Definição
- História
- Tipos de Paradigmas
- Exemplos de Aplicações
- Conclusão



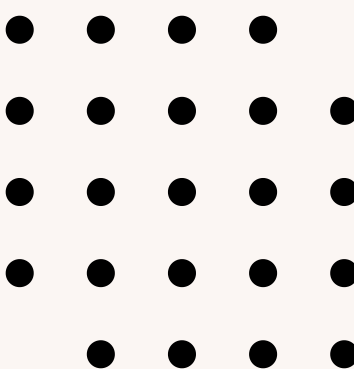
DEFINIÇÃO

- Modelo padrão de execução de um programa.
- Define como será a execução e a estrutura do programa.
- Diversas linguagens de programação utilizam diferentes conjuntos de instruções (paradigmas).
- Algumas linguagens suportam múltiplos paradigmas, enquanto outras são voltadas para paradigmas específicos.



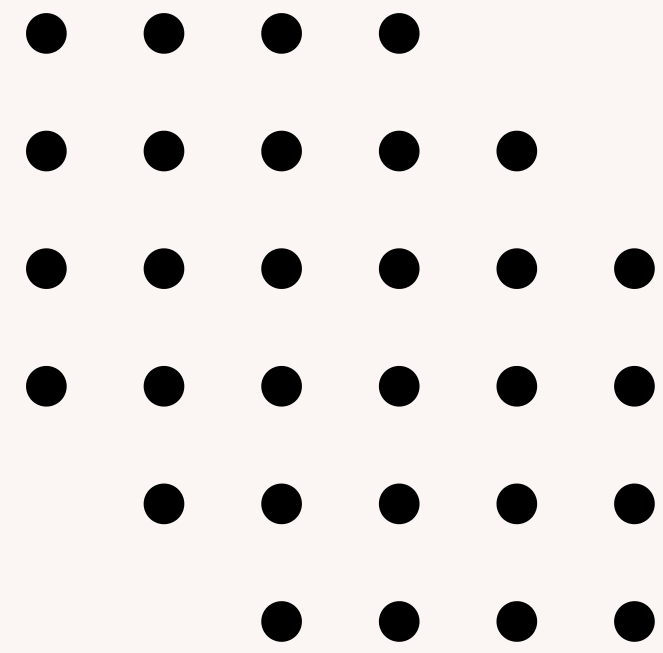
HISTÓRIA

- Surgiram nos primórdios da computação.
- Evoluíram ao longo do tempo para atender demandas por expressividade, abstração e eficiência.
- Exemplos incluem o paradigma imperativo, orientado a objetos e funcional.
- Novos paradigmas foram desenvolvidos, como a programação lógica, concorrente e baseada em eventos.
- A evolução dos paradigmas foi influenciada pelo avanço da tecnologia e pelas necessidades dos desenvolvedores.



TIPOS DE PARADIGMAS

- Imperativa
- Lógica
- Orientado à objetos
- Procedural
- Funcional
- Declarativa
- Orientado à eventos
- Concorrente



IMPERATIVO

- Estrutura em um conjunto de sequências de instruções diretas
- Manipulações de dados de estoque

EXEMPLO DE PROGRAMAÇÃO IMPERATIVA EM PYTHON

```
DEF CALCULAR_MEDIA(LISTA):
```

```
    SOMA = 0
```

```
    FOR ELEMENTO IN LISTA:
```

```
        SOMA += ELEMENTO
```

```
    MEDIA = SOMA / LEN(LISTA)
```

```
    RETURN MEDIA
```

```
NOTAS = [8, 7, 9, 6, 8]
```

```
MEDIA_NOTAS = CALCULAR_MEDIA(NOTAS)
```

```
PRINT("A MÉDIA DAS NOTAS É:", MEDIA_NOTAS)
```



LÓGICO

Um exemplo prático de uso da programação lógica pode ser encontrado em sistemas de diagnóstico médico, onde o programa é capaz de inferir doenças com base nos sintomas relatados pelo paciente.

- Estrutura com base nas regras de lógica e inferências
- Pouco utilizada, mais para manipulações

```
# EXEMPLO DE PROGRAMAÇÃO LÓGICA EM PYTHON COM PYDATALOG
FROM PYDATALOG IMPORT PYDATALOG

PYDATALOG.CREATE_TERMS('X, Y')

# REGRAS LÓGICAS
+ (X == 2)
Y == X * 3

PRINT("O VALOR DE Y É:", Y)
# RESULTADO: Y = 6
```

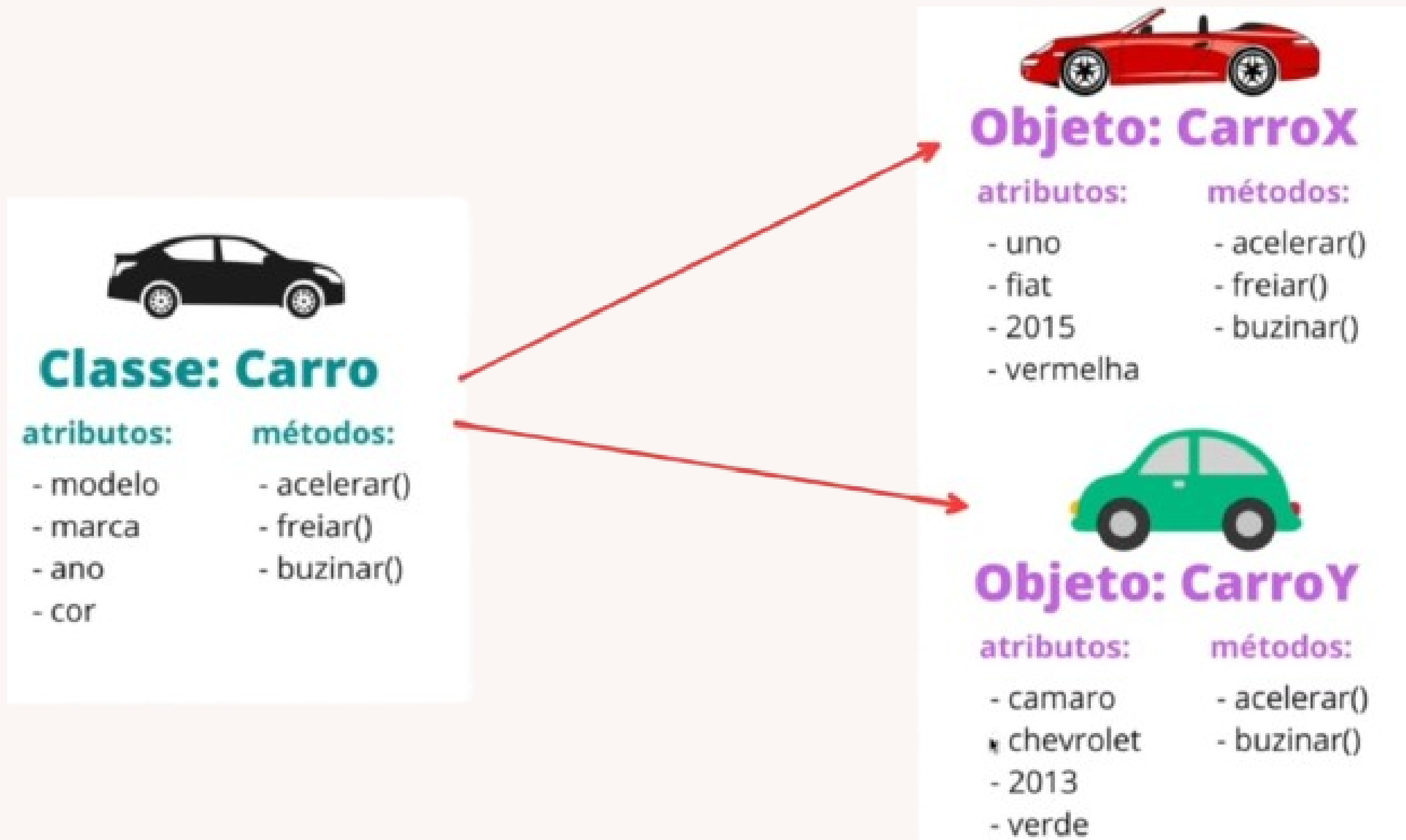


ORIENTADO À OBJETOS



- Classe: Uma classe é um modelo ou molde que define as características e comportamentos de um grupo de objetos semelhantes.
- Objetos: Um objeto é uma instância específica criada a partir de uma classe.

EXEMPLOS



PROCEDURAL



- Sequencial: As operações são executadas em ordem, passo a passo.
- Reutilização de código: É possível reutilizar funções em vários pontos do programa.
- Fácil depuração: Facilita a identificação de erros através da execução linear do código.

EXEMPLOS

- Sequencialidade: sequencial, Assim como em uma receita de bolo, onde cada etapa segue a anterior.
- Reutilização de código: Imagine que você tem uma receita básica para um bolo de baunilha. Você pode reutilizar essa base para criar diferentes variações, como um bolo de chocolate ou um bolo de morango.

PROGRAMAÇÃO FUNCIONAL



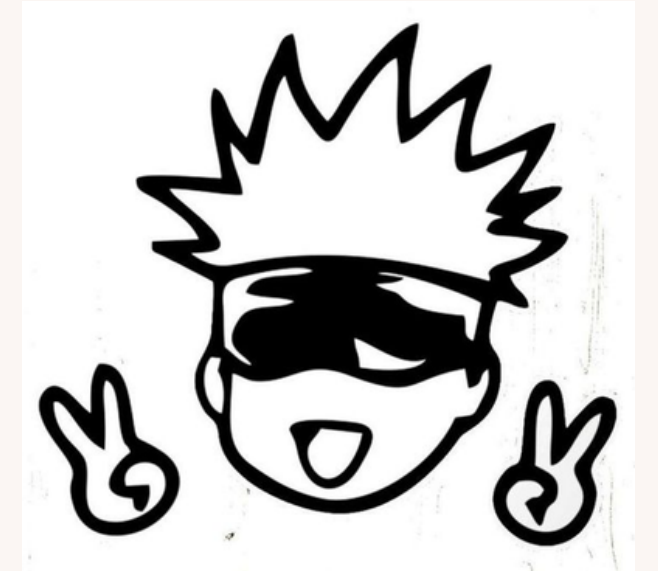
- A programação funcional se concentra na avaliação de expressões e na aplicação de funções matemáticas.
- Ela trata a computação como a avaliação de funções.
- Evita o uso de estados mutáveis e dados compartilhados.

EXEMPLOS

```
DEF DOBRAR_ELEMENTOS(LISTA):  
    RETURN LIST(MAP(LAMBDA X: X * 2, LISTA))
```

```
NUMEROS = [1, 2, 3, 4, 5]  
DOBRO = DOBRAR_ELEMENTOS(NUMEROS)  
PRINT("O DOBRO DOS NÚMEROS É:", DOBRO)
```


PROGRAMAÇÃO DECLARATIVA



- Envolve expressar a lógica do problema em termos de restrições, regras e relações entre os dados.
- Sem se preocupar com a sequência de comandos para alcançar o resultado desejado.
- É associada diretamente com bancos de dados.
- O programa descreve o que deve ser feito, muito comum ser utilizado em Sistemas Gerenciadores de Banco de Dados (SGBD):

EXEMPLOS DE PROGRAMAÇÃO DECLARATIVA EM SQL:

```
//SELECIONE OS ATRIBUTOS/CAMPOS
```

```
SELECT NOME, IDADE
```

```
//NA COLEÇÃO/TABELA
```

```
FROM ESCOLA SUPERIOR
```

```
//PARA TODOS CADASTROS QUE CIDADE – TÓQUIO
```

```
WHERE CIDADE = 'TOKYO'
```

```
//SELECIONE DOCENTE, DISCENTE = PESSOAS
```

```
SELECT *
```

```
FROM PESSOAS
```

```
WHERE TIPO = 'DOCENTE' OR TIPO = 'DICENTE';
```

```
//SELECIONE GRAU DE ESPECIALIZAÇÃO
```

```
SELECT *
```

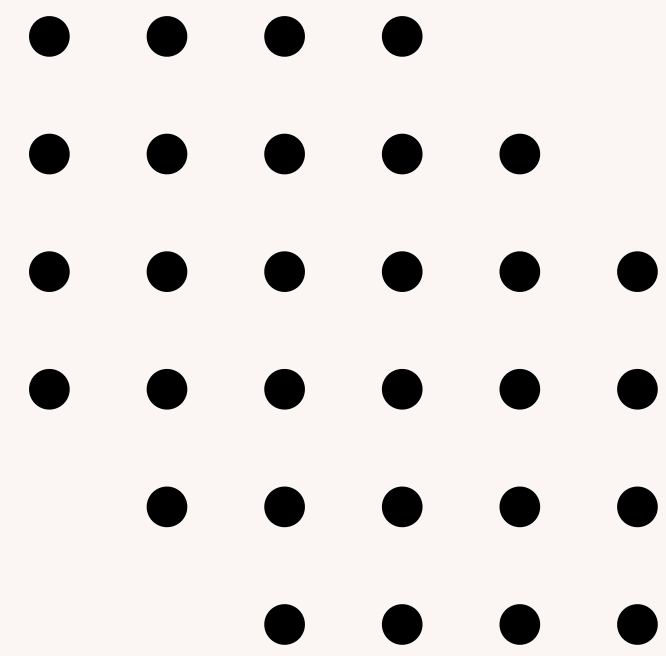
```
FROM PESSOAS
```

```
WHERE GRAU IN ('GRAU 1', 'GRAU 2', 'GRAU 3',  
'GRAU 4');
```

PROGRAMAÇÃO ORIENTADA A EVENTOS



- É um paradigma de programação que se baseia no conceito de eventos
- O programa responde a eventos que ocorrem em tempo real;
- Controla o fluxo do programa;
- Exemplo: Ocorre durante a execução do programa, como um clique de mouse



EXEMPLOS

```
IMPORT TKINTER AS TK
```

```
DEF CLIQUE_BOTAO():  
    LABEL.CONFIG(TEXT="BOTÃO CLICADO!")
```

```
ROOT = TK.TK()  
ROOT.GEOMETRY("200X100")
```

```
BOTAO = TK.BUTTON(ROOT, TEXT="CLIQUE AQUI", COMMAND=CLIQUE_BOTAO)  
BOTAO.PACK()
```

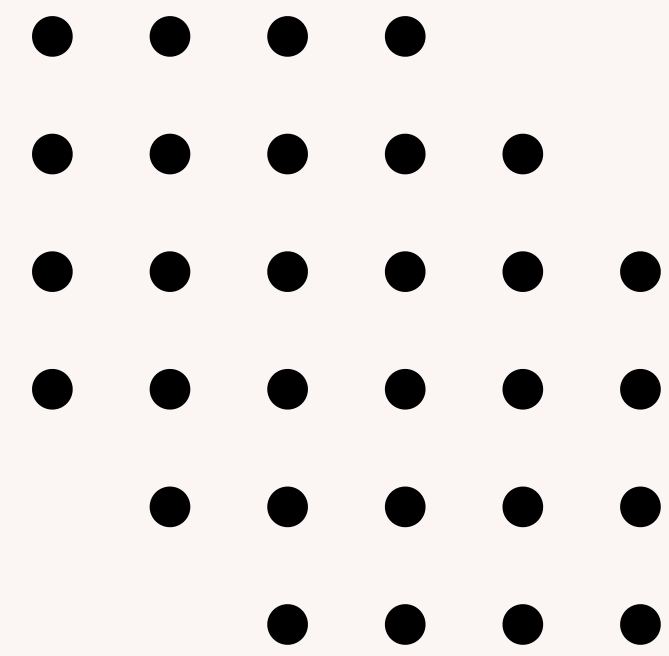
```
LABEL = TK.LABEL(ROOT, TEXT="")  
LABEL.PACK()
```

```
ROOT.MAINLOOP()
```

PROGRAMAÇÃO CONCORRENTE



- É um paradigma que várias tarefas são executadas simultaneamente;
- Partes do programa podem ser executadas em paralelo;
- O objetivo é melhorar o desempenho e a eficiência do programa
- Exemplo: Threading em Python;



EXEMPLOS

```
import threading
import time

def imprimir_numeros():
    for i in range(1, 6):
        print("Número:", i)
        time.sleep(1)

def imprimir_letras():
    for letra in 'ABCDE':
        print("Letra:", letra)
        time.sleep(1)

# criando duas threads
thread1 = threading.Thread(target=imprimir_numeros)
thread2 = threading.Thread(target=imprimir_letras)

# iniciando as threads
thread1.start()
thread2.start()

# aguardando as threads terminarem
thread1.join()
thread2.join()
```

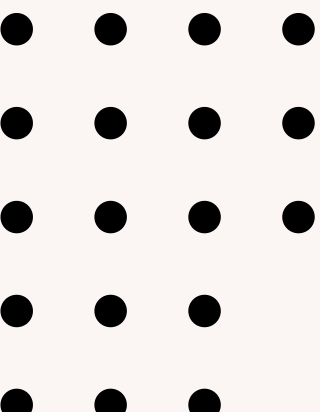
EXEMPLOS DE APLICAÇÕES

- **Programação Imperativa:**

- Desenvolvimento de sistemas de controle de estoque em empresas.
- Operações como adicionar ou remover itens, atualizar quantidades e calcular o valor total são comuns.

- **Programação Lógica:**

- Sistemas de diagnóstico médico:
- Utilização de regras lógicas para inferir doenças com base nos sintomas informados pelo paciente.
- As regras lógicas são aplicadas para determinar as possíveis condições de saúde do paciente, oferecendo sugestões de diagnóstico e tratamento.



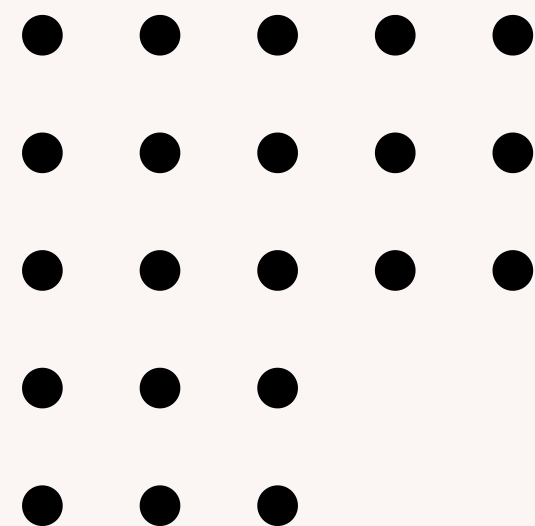
EXEMPLOS DE APLICAÇÕES

- **Programação Funcional:**

- Sistemas de processamento de dados financeiros.
- Processamento de transações financeiras, análise de mercado, cálculo de indicadores financeiros, entre outros.

- **Programação Declarativa – Banco de Dados:**

- Sistemas de gerenciamento de banco de dados.
- Utilização de consultas SQL para recuperar e manipular dados.
- Declaração das condições que os dados devem satisfazer, em vez de especificar passo a passo como os dados devem ser acessados.
- Consulta de informações de clientes, geração de relatórios, análise de tendências de mercado, entre outros.



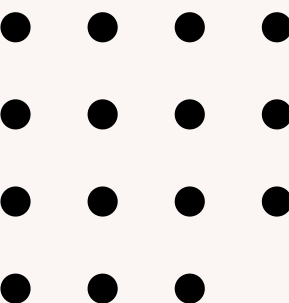
EXEMPLOS DE APLICAÇÕES

- **Programação Orientada a Eventos:**

- Sistemas de monitoramento de tráfego de rede.
- Tratamento assíncrono dos eventos permite uma resposta rápida e eficaz.
- Monitoramento de tráfego de rede em empresas, detecção de intrusões em sistemas de segurança, controle de dispositivos IoT, entre outros.

- **Programação Concorrente:**

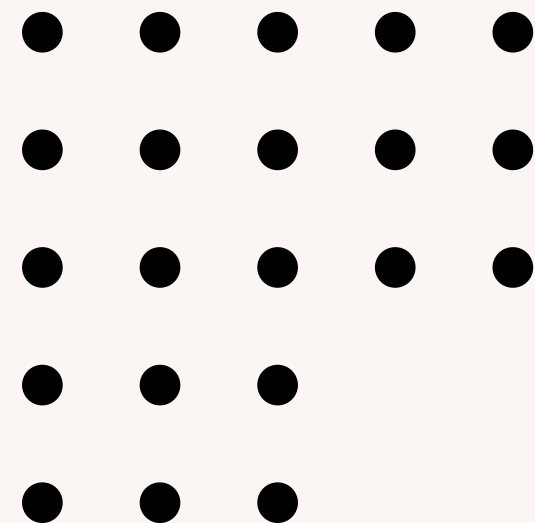
- Sistemas de processamento de transações financeiras em tempo real.
- Utilização da programação concorrente para processar múltiplas transações simultaneamente.
- Garantia de uma resposta rápida e evita atrasos no processamento.
- Processamento de transações bancárias, negociação de ações na bolsa de valores, processamento de pedidos em comércio eletrônico, entre outros.



CONCLUSÃO

Objetivos:

- Compreender o significado de paradigmas.
- Saber a origem dos paradigmas.
- Entender alguns dos mais famosos e úteis.
- Visualizar exemplos de aplicações baseados nos tipos de paradigmas.



REFERÊNCIAS BIBLIOGRÁFICAS

1. Paradigmas de Programação. Disponível em:
 <<https://guia.dev/pt/pillars/languages-and-tools/programming-paradigms.html>>. Acesso em: 10 mar. 2024.

2. SEBESTA, R. W. Conceitos de Linguagens de Programação – 11.ed. [s.l.] Bookman Editora, 2018.

3. TUCKER, A.; NOONAN, R. Linguagens de Programação – 2.ed.: Princípios e Paradigmas. [s.l: s.n.].

4. MALAQUIAS, J. R. Paradigmas de Programação. [s.d.].

5. Algoritmos e técnicas de programação. [s.l.] Editora e Distribuidora Educacional S.A., 2021.

